Esercitazione: griglia di oggetti e animazione 2D

OpenGL, matrici di modellazione e Dear ImGui

Obiettivo. Realizzare una scena 2D composta da farfalle e cuori disposti in una griglia configurabile e successivamente animarla, aggiornando le matrici di modellazione degli oggetti in funzione del tempo. L'attività prosegue il lavoro svolto sulle classi `ShapeGenerator`, `Geometry`, `Scene`, `Renderer`, `Shader` e `Gui`.

Materiale di partenza e impostazione

Utilizzare il progetto OpenGL a classi fornito. Si assuma una proiezione ortografica che visualizza una regione del mondo 2D comprendente il quadrato $[-100, 100] \times [-100, 100]$. Le forme devono essere generate nel proprio sistema di riferimento locale; la loro posizione e dimensione nella scena devono essere controllate tramite una matrice di modellazione.
Indicazione generale. Separare la *creazione* della griglia dall'*aggiornamento* dell'animazione: le geometrie e i buffer GPU non devono essere ricostruiti a ogni frame.

1 Esercizio 1 — Costruzione di una griglia configurabile

Scrivere il codice necessario a disegnare una griglia di oggetti organizzati in righe e colonne, rispettando i seguenti requisiti.

1. Il numero di righe `numRows` e di colonne `numCols` deve poter essere modificato dall'utente mediante due controlli `ImGui::SliderInt()`.
2. Le righe con indice pari $(0, 2, 4, \ldots)$ devono contenere **farfalle**; quelle con indice dispari $(1, 3, 5, \ldots)$ devono contenere **cuori**. Si consideri la prima riga, in alto, con indice 0.
3. Ogni oggetto deve essere collocato al centro della cella individuata dalla propria riga e colonna, con una scala che lo mantenga contenuto nella cella.
4. La griglia deve occupare inizialmente **la metà superiore** della regione visibile, lasciando libera la metà inferiore.
5. La modifica del numero di righe o colonne deve aggiornare correttamente la disposizione degli oggetti.

Come procedere

a) Aggiungere alla classe `Gui` due membri privati interi per il numero di righe e colonne, con opportuni metodi pubblici di lettura. Inserire gli slider nel metodo che costruisce l'interfaccia.

b) Definire una funzione per costruire la griglia. Utilizzare due cicli annidati, uno per le righe e uno per le colonne; selezionare la forma con il test `row % 2 == 0`.

c) Scelti i limiti della griglia L, R, B, T, calcolare la larghezza e l'altezza delle celle:

$$\Delta x = \frac{R - L}{\texttt{numCols}}, \qquad \Delta y = \frac{T - B}{\texttt{numRows}}.$$

Il centro della cella di riga r e colonna c si ottiene con

$$x_{r,c} = L + \left(c + \frac{1}{2}\right)\Delta x, \qquad y_{r,c} = T - \left(r + \frac{1}{2}\right)\Delta y.$$



Figura 1: .

d) Per ogni oggetto costruire una Model Matrix che applichi una scala e una traslazione:

$$M_{r,c} = T(x_{r,c}, y_{r,c})\, S(s_{x,r,c}, s_{y,r,c}).$$

Ricordare che, con vettori colonna, la scala viene applicata prima della traslazione.

e) Evitare di ricreare la scena a ogni iterazione del render loop: ricostruirla solo quando `numRows` o `numCols` cambiano.

Verifica. Provare almeno tre configurazioni, ad esempio 2×3, 4×5 e 6×4. Controllare l'alternanza delle forme, il corretto centraggio nelle celle e l'assenza di sovrapposizioni significative.

2 Esercizio 2 — Animazione ondulatoria e discesa

A partire dalla griglia realizzata nell'esercizio precedente, introdurre un'animazione con le seguenti caratteristiche.

1. Ogni oggetto oscilla orizzontalmente con moto sinusoidale. La frequenza e la fase dipendono dall'indice di riga, così da generare un effetto ondulatorio.
2. La griglia scende progressivamente verso il basso con velocità costante. Per ottenere una discesa *continua*, prevedere la ricomparsa dall'alto degli oggetti usciti dalla regione visibile.
3. A ogni frame aggiornare le matrici di modellazione delle singole geometrie; non modificare le posizioni originali dei vertici né trasferire nuovamente i VBO.

Modello matematico suggerito

Siano $(x^0_{r,c}, y^0_{r,c})$ le coordinate iniziali del centro della cella. Definire

$$f_r = f_0 + r\,\Delta f, \qquad \varphi_r = r\,\Delta\varphi,$$
$$x_{r,c}(t) = x^0_{r,c} + A\sin(2\pi f_r t + \varphi_r), \qquad y_{r,c}(t) = y^0_{r,c} - vt.$$

Qui A è l'ampiezza dell'oscillazione, f_0 la frequenza iniziale, Δf l'incremento per riga, $\Delta\varphi$ lo sfasamento tra righe e v la velocità di discesa. Il tempo t può essere letto tramite `glfwGetTime()`.
Nota sulla discesa continua. La legge $y(t) = y^0 - vt$ da sola porta gli oggetti definitivamente fuori dalla finestra. Per la ricomparsa dall'alto occorre aggiungere una regola di riciclo della posizione verticale, mantenendo la distanza tra le righe. È possibile iniziare verificando il moto senza riciclo e introdurre successivamente la periodicità.

Come procedere

a) Memorizzare per ogni oggetto l'indice nella `Scene`, l'indice della riga, il centro iniziale e i fattori di scala. Una struttura C++ dedicata può facilitare questa gestione.

b) Scrivere una funzione di aggiornamento dell'animazione che riceva la scena, le informazioni sugli oggetti e il tempo corrente.

c) Per ogni oggetto calcolare la posizione animata e ricostruire la matrice

$$M_{r,c}(t) = T(x_{r,c}(t), y_{r,c}(t))\, S_{r,c}.$$

d) Assegnare la matrice alla geometria mediante `setModelMatrix()`. La funzione di aggiornamento deve essere richiamata *prima* di `renderer.drawScene(scene)`.

e) Sperimentare valori differenti dei parametri A, f_0, Δf, $\Delta\varphi$ e v per ottenere un moto leggibile e regolare.